A `KdTree` just contains a root node, and a constructor method, taking an optional array of points as input. We'll take a look at how a k-d tree can be constructed later, after we introduce the insertion method. For now, it will suffice to say that it will set the root to either a void entry (being it `null`, a special instance of `KdNode`, or whatever it's more appropriate for the language used).

For the sake of convenience, let's assume a tree also stores the value `k`, the dimension of the space on which the tree is defined.

The root is, as said, an instance of `KdNode`: this data structure models a node of a *BST*, with its left and right children, and its value, a point in the k-dimensional space. We will use the special value `null` to model an empty node (and thus an empty tree).

9.3.1 Search

In section 9.2 we have implicitly described how search works on k-d trees. There is nothing fancy in this algorithm, it's just a regular search on a binary search trees storing tuples, with the caveat that instead of comparing the whole tuple at each step, we only use one coordinate: at level `i`, we compare the `i`-th coordinate (or `i mod k`, if `i`≥`k`).

Listing 9.2 Helper functions

```
function
getNodeKey(node)
          #1
```